Predicting Medical Expenses for an Insurance Company Using Multiple Linear Regression

Introduction:

In this activity, we will explore the concept of linear regression using the `insurance.csv` dataset. Linear regression is a supervised learning technique used to predict the value of a dependent variable (in this case, `charges`, based on one or more independent variables (for example, `age`, `bmi`, `children`, etc.).

You can download the dataset from here.

The `insurance.csv` dataset contains information about individuals and their medical charges. Each of the variables in the file is described below:

1. **age**: The individual's age (in years).
2. **sex**: The individual's sex (male for men and female for women).
3. **bmi**: The individual's Body Mass Index (BMI), a measure of weight relative to height.
4. **children**: Number of children or dependents the individual has.
5. **smoker**: Indicates whether the individual is a smoker (yes or no).
6. **region**: Geographic region where the individual resides (northeast, northwest, southeast, southwest).
7. **charges**: Total medical charges billed to the individual (in dollars).

These variables are commonly used to analyze and predict medical costs based on individuals' demographic and health characteristics.

Contents:

Import the required packages

In [1]:
```
# Install the required packages
%pip install scikit-learn # Machine learning package with classification, regres
%pip install pandas
%pip install matplotlib
%pip install seaborn
%pip install numpy

# Import the required packages
from sklearn.linear_model import LinearRegression # Linear regression with sciki
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import numpy as np
```

```
Note: you may need to restart the kernel to use updated packages.
ERROR: Invalid requirement: '#': Expected package name at the start of dependency
specifier
    #
    ^
```

```
Requirement already satisfied: pandas in c:\RD\2026_2027\M3V2\.venv\Lib\site-pack
ages (3.0.6)
Requirement already satisfied: numpy>=1.26.0 in c:\RD\2026_2027\M3V2\.venv\Lib\si
te-packages (from pandas) (2.5.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\RD\2026_2027\M3V2\.ve
nv\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\RD\2026_2027\M3V2\.venv\Lib\site-pack
ages (from pandas) (2026.5)
Requirement already satisfied: six>=1.5 in c:\RD\2026_2027\M3V2\.venv\Lib\site-pa
ckages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: matplotlib in c:\RD\2026_2027\M3V2\.venv\Lib\site-
packages (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\RD\2026_2027\M3V2\.venv\Lib
\site-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in c:\RD\2026_2027\M3V2\.venv\Lib\sit
e-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in c:\RD\2026_2027\M3V2\.venv\Li
b\site-packages (from matplotlib) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\RD\2026_2027\M3V2\.venv\Li
b\site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: numpy>=1.25 in c:\RD\2026_2027\M3V2\.venv\Lib\site
-packages (from matplotlib) (2.5.4)
Requirement already satisfied: packaging>=20.0 in c:\RD\2026_2027\M3V2\.venv\Lib
\site-packages (from matplotlib) (26.3)
Requirement already satisfied: pillow>=9 in c:\RD\2026_2027\M3V2\.venv\Lib\site-p
ackages (from matplotlib) (13.0.0)
Requirement already satisfied: pyparsing>=3 in c:\RD\2026_2027\M3V2\.venv\Lib\sit
e-packages (from matplotlib) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in c:\RD\2026_2027\M3V2\.venv
\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\RD\2026_2027\M3V2\.venv\Lib\site-pa
ckages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: seaborn in c:\RD\2026_2027\M3V2\.venv\Lib\site-pac
kages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\RD\2026_2027\M3V2\.venv
\Lib\site-packages (from seaborn) (2.5.4)
Requirement already satisfied: pandas>=1.2 in c:\RD\2026_2027\M3V2\.venv\Lib\site
-packages (from seaborn) (3.0.6)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\RD\2026_2027\M3V2\.v
env\Lib\site-packages (from seaborn) (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\RD\2026_2027\M3V2\.venv\Lib
\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.0)
Requirement already satisfied: cycler>=0.10 in c:\RD\2026_2027\M3V2\.venv\Lib\sit
e-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in c:\RD\2026_2027\M3V2\.venv\Li
b\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.67.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\RD\2026_2027\M3V2\.venv\Li
b\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.1)
Requirement already satisfied: packaging>=20.0 in c:\RD\2026_2027\M3V2\.venv\Lib
\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.3)
Requirement already satisfied: pillow>=9 in c:\RD\2026_2027\M3V2\.venv\Lib\site-p
ackages (from matplotlib!=3.6.1,>=3.4->seaborn) (13.0.0)
Requirement already satisfied: pyparsing>=3 in c:\RD\2026_2027\M3V2\.venv\Lib\sit
e-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in c:\RD\2026_2027\M3V2\.venv
\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\RD\2026_2027\M3V2\.venv\Lib\site-pack
ages (from pandas>=1.2->seaborn) (2026.5)
```

```
Requirement already satisfied: six>=1.5 in c:\RD\2026_2027\M3V2\.venv\Lib\site-pa
ckages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: numpy in c:\RD\2026_2027\M3V2\.venv\Lib\site-packa
ges (2.5.4)
Note: you may need to restart the kernel to use updated packages.
```

Load the dataset

In [2]:
```
# Load the dataset directly from the GitHub repository
insurance = pd.read_csv('https://raw.githubusercontent.com/eugeniomorocho/Artifi
insurance.head(10)
```

Out[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
5	31	female	25.740	0	no	southeast	3756.62160
6	46	female	33.440	1	no	southeast	8240.58960
7	37	female	27.740	3	no	northwest	7281.50560
8	37	male	29.830	2	no	northeast	6406.41070
9	60	female	25.840	0	no	northwest	28923.13692

In [3]:
```
# Print the unique values of 'region'
insurance['region'].unique()
```

Out[3]:
```
<StringArray>
['southwest', 'southeast', 'northwest', 'northeast']
Length: 4, dtype: str
```

Data preprocessing

Why use dummy variables for region instead of a numeric mapping?

Linear regression needs numeric inputs, but converting a category to a number also introduces assumptions. `region` is **nominal**: northeast, northwest, southeast, and southwest have no natural ranking.

Encoding	Advantages	Disadvantages
Map regions to 0, 1, 2, 3	Short code and only one column	Imposes an artificial order and equal spacing. One coefficient forces the same change in predicted charges between consecutive codes; changing the assigned numbers can change the fitted predictions.

Encoding	Advantages	Disadvantages
Dummy variables (one-hot encoding)	Represents each region with a separate binary indicator without assuming an order	Adds columns and requires consistent encoding when predicting new cases. Many categories can create a large feature matrix.

For example, mapping northeast → 0, northwest → 1, southeast → 2, and southwest → 3 would force the southwest-versus-northeast contribution to be three times the northwest-versus-northeast contribution. There is no reason to impose that relationship.

For this explanation exercise, we keep **all four region columns** with `drop_first=False`, so every region appears in the coefficient and contribution plots. Each record has exactly one active region indicator. No region is omitted or used as a reference category: `northeast`, `northwest`, `southeast`, and `southwest` each have their own column.

With an intercept, these four columns are redundant: their sum is always one. The individual region coefficients and intercept are therefore not uniquely identifiable. The least-squares solver selects a numerical solution; different coefficient representations can give the same predictions on valid records.

Mapping works well for the binary features here: `smoker` and `sex` have only two categories, so their single coefficient represents a difference between two groups. For a genuinely ordered feature, numeric mapping may be reasonable, but a linear model still assumes equal effects per code step. For `region`, dummy encoding is the clearer choice.

In [4]:
```
# Encode the categorical variables 'sex' and 'smoker' by replacing labels (label
insurance['sex'] = insurance['sex'].replace({'female': 0, 'male': 1})
insurance['smoker'] = insurance['smoker'].replace({'no': 0, 'yes': 1})

# Keep all four region indicators visible for the explanation exercise
insurance = pd.get_dummies(insurance, columns=['region'], drop_first=False, dtyp

# Display the first 10 rows after preprocessing
insurance.head(10)
```

Out[4]:

	age	sex	bmi	children	smoker	charges	region_northeast	region_northwest
0	19	0	27.900	0	1	16884.92400	0	0
1	18	1	33.770	1	0	1725.55230	0	0
2	28	1	33.000	3	0	4449.46200	0	0
3	33	1	22.705	0	0	21984.47061	0	1
4	32	1	28.880	0	0	3866.85520	0	1
5	31	0	25.740	0	0	3756.62160	0	0
6	46	0	33.440	1	0	8240.58960	0	0
7	37	0	27.740	3	0	7281.50560	0	1
8	37	1	29.830	2	0	6406.41070	1	0
9	60	0	25.840	0	0	28923.13692	0	1

Prepare the variables `X` and `y`

In [5]:
```
# Select the independent variables (X)
X = insurance.drop('charges', axis=1)
X
```

Out[5]:

	age	sex	bmi	children	smoker	region_northeast	region_northwest	region_s
0	19	0	27.900	0	1	0	0	
1	18	1	33.770	1	0	0	0	
2	28	1	33.000	3	0	0	0	
3	33	1	22.705	0	0	0	1	
4	32	1	28.880	0	0	0	1	
...	...	...	...	...	...	...	...	
1333	50	1	30.970	3	0	0	1	
1334	18	0	31.920	0	0	1	0	
1335	18	0	36.850	0	0	0	0	
1336	21	0	25.800	0	0	0	0	
1337	61	0	29.070	0	1	0	1	

1338 rows × 9 columns

In [6]:
```
# Select the dependent variable (y)
y = insurance['charges']
y
```

Out[6]:
```
0       16884.92400
1        1725.55230
2        4449.46200
3       21984.47061
4        3866.85520
            ...
1333    10600.54830
1334     2205.98080
1335     1629.83350
1336     2007.94500
1337    29141.36030
Name: charges, Length: 1338, dtype: float64
```

Split the data into training and test sets

In [7]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

Instantiate a Linear Regression model and fit it with the values of `X_train` and `y_train`

In [8]:
```
# Create an instance of the linear regression model
regression_model = LinearRegression()

# Fit the model to the training data
regression_model.fit(X_train, y_train)
```

Out[8]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

(Optional) Save the model in .pkl format

The `.pkl` format is a file extension used to store serialized objects in Python through the `pickle` module. This module allows Python objects to be converted into a sequence of bytes, making it easier to store them in files or transmit them over networks. `.pkl` files are commonly used to save trained machine learning models, complex data structures, or any other object that needs to persist between program execution sessions. Deserializing these files allows objects to be restored to their original state, keeping their structure and data intact.

In [9]:
```
# Save the model for future use
import joblib
joblib.dump(regression_model, 'regression_model.pkl')
```

Out[9]:
```
['regression_model.pkl']
```

(Optional) Load the model in .pkl format

In [10]:
```
# Load the saved model
regression_model = joblib.load('regression_model.pkl')
```

Get the coefficients and intercept of the regression model

In the context of linear regression, the coefficients and intercept are fundamental parameters that define the relationship between the independent (predictor) variables and the dependent (response) variable.

a) Coefficients

The coefficients (also known as weights or betas) represent the magnitude and direction of the relationship between each independent variable and the dependent variable. In a multiple linear regression equation, each coefficient indicates how much the dependent variable changes when the corresponding independent variable increases by one unit, holding the other independent variables constant. Mathematically, if we have a linear regression equation of the form:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n$$

where:

- y is the dependent (target) variable.
- β_0 is the intercept.
- $\beta_1, \beta_2, \ldots, \beta_n$ are the coefficients of the independent variables $X_1, X_2, \ldots, X_n$.

b) Intercept

The intercept β_0 is the value of the dependent variable when all the independent variables are equal to zero. It represents the point where the regression line crosses the y axis. In other words, it is the predicted value of y when there is no influence from the independent variables.

Example:

If we have a multiple linear regression equation with two independent variables, it could look like this:

$$y = 3 + 2X_1 - 4X_2$$

In this case:

- The intercept β_0 is 3.
- The coefficient of X_1 (β_1) is 2, which means that for each one-unit increase in X_1, y increases by 2 units.
- The coefficient of X_2 (β_2) is -4, which means that for each one-unit increase in X_2, y decreases by 4 units.

These parameters are essential for interpreting the linear regression model and understanding how the independent variables affect the dependent variable.

In [11]:
```
# Get the coefficients
coefficients = regression_model.coef_
coefficients
```

Out[11]:
```
array([ 2.56975706e+02, -1.85916916e+01,  3.37092552e+02,  4.25278784e+02,
        2.36511289e+04,  4.59585244e+02,  8.89079180e+01, -1.98279052e+02,
       -3.50214110e+02])
```

In [12]:
```
# Get the intercept
intercept = regression_model.intercept_
intercept
```

Out[12]:
```
np.float64(-12390.804294570278)
```

In [13]:
```
# Define the regression line equation
line_equation = 'y = ' + ' + '.join([f'{coefficients[i]}*X{i}' for i in range(le
print(f'Regression line equation: {line_equation}')
```

```
Regression line equation: y = 256.97570583119455*X0 + -18.591691641004672*X1 + 33
7.09255194877466*X2 + 425.27878352427877*X3 + 23651.128855761235*X4 + 459.5852442
437305*X5 + 88.90791801592536*X6 + -198.279052320906*X7 + -350.2141099387532*X8 +
-12390.804294570278
```

With the regression line equation, we can make predictions about medical charges based on the input features. This equation allows us to understand how each feature influences the final outcome, providing a clear mathematical relationship between the independent variables (features) and the dependent variable (medical charges). By applying this equation to new data, we can estimate the expected medical charges, which is useful for predictive analysis, informed decision-making, and resource planning in healthcare. In addition, the regression line equation makes it easier to interpret the coefficients, helping us identify which factors have the greatest impact on medical costs.

The coefficients represent the expected change in medical charges for a one-unit increase in the corresponding independent variable, holding the other variables constant.

Their interpretation depends on the variable's units and encoding:

1. **Numerical variables**: A coefficient describes the change in predicted charges per unit of the variable. Raw coefficient magnitudes cannot be compared directly across variables measured on different scales.
2. **Binary variables**: A coefficient describes the difference in predicted charges between the category coded as 1 and the category coded as 0, holding the other variables constant. For region, all four indicators are included. Switching from region A to region B changes two indicators, so the predicted difference is the coefficient of B minus the coefficient of A. A single region coefficient is not a uniquely identified effect because the model also has an intercept.

The sign indicates the direction of the fitted association:

- **Positive coefficient**: A higher value is associated with higher predicted medical charges.
- **Negative coefficient**: A higher value is associated with lower predicted medical charges.

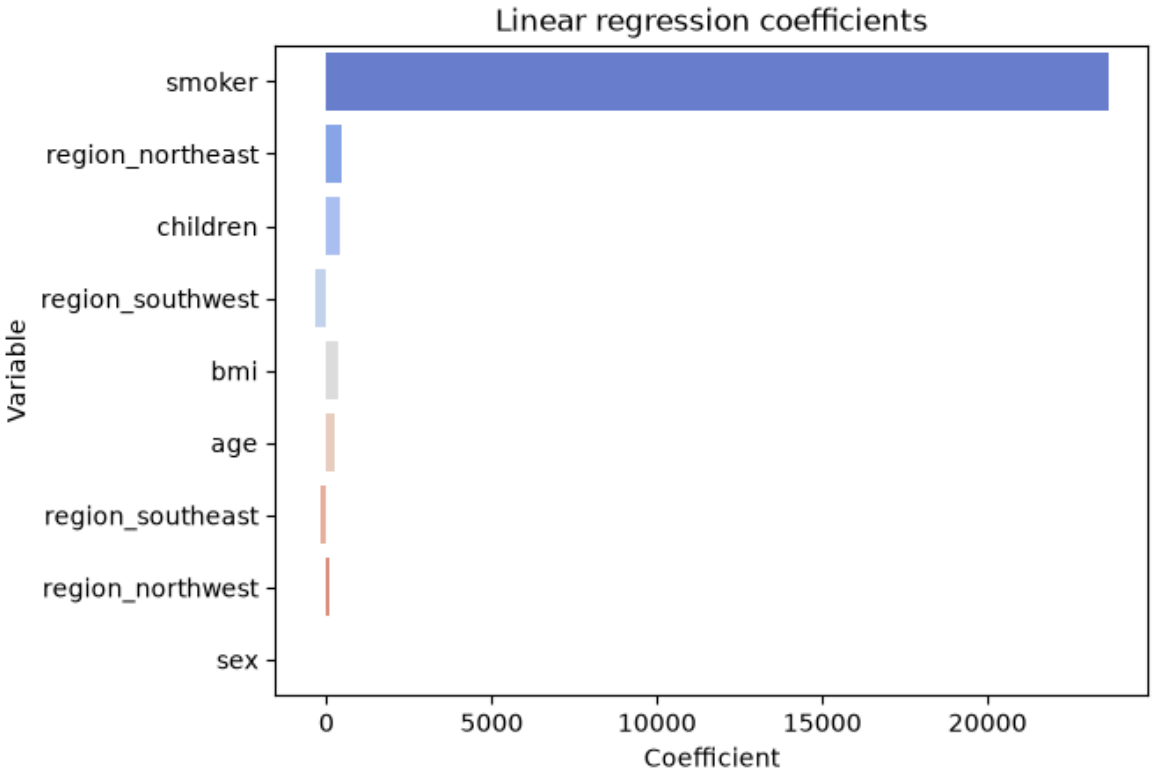
Standardizing the predictors makes coefficient magnitudes more comparable in terms of a one-standard-deviation change. However, correlated predictors can make coefficients unstable, and coefficient magnitude alone does not establish predictive importance, statistical significance, or causality.

In [14]:
```
# Create a DataFrame to make interpretation easier
coef_df = pd.DataFrame({'Variable': X.columns, 'Coefficient': coefficients})

# Sort by absolute coefficient magnitude; the predictors have different units
coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()

# Sort the coefficients from highest to lowest
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False)

print(coef_df)
```

```
          Variable   Coefficient  Abs_Coefficient
4           smoker  23651.128856     23651.128856
5  region_northeast    459.585244       459.585244
3         children    425.278784       425.278784
8  region_southwest   -350.214110       350.214110
2              bmi    337.092552       337.092552
0              age    256.975706       256.975706
7  region_southeast   -198.279052       198.279052
6  region_northwest     88.907918        88.907918
1              sex    -18.591692        18.591692
```

In [15]:
```
# Plot the coefficients sorted from highest to lowest in a bar chart
sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')

plt.xlabel('Coefficient')
plt.ylabel('Variable')
plt.title('Linear regression coefficients')
plt.show()
```

```
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_28508\1143019381.py:2: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')
```



Do smaller coefficients mean that age and BMI are unimportant?

No. The model uses both variables, but the raw coefficient chart compares different units:

- The `age` coefficient is the predicted change in dollars for **one additional year**, holding the other predictors fixed.
- The `bmi` coefficient is the predicted change in dollars for **one BMI unit**, holding the other predictors fixed.
- The `smoker` coefficient represents the **entire change from nonsmoker (0) to smoker (1)**.

A one-year change is not directly comparable to a change in smoking status. Across ten years, the fitted age contribution changes by $10\beta_{\text{age}}$; across five BMI units, it changes by $5\beta_{\text{bmi}}$. Thus, relatively small per-unit coefficients can accumulate into substantial differences across the observed feature range.

A large smoking coefficient can also visually compress the other bars. The standardized-coefficient section below compares changes of one standard deviation, and the final SHAP section compares contributions in dollars across the test observations.

This additive model also assigns one constant slope to age and one to BMI. It does not include nonlinear effects or a BMI-by-smoking interaction. If those relationships matter, the current model may miss them; a small coefficient is not evidence that the feature has no predictive value. Neither coefficients nor SHAP values establish a causal health effect.

Calculate the model evaluation metrics (making predictions on the test set)

In the context of linear regression, it is essential to evaluate the model's performance to understand its accuracy and generalization ability. There are several evaluation metrics that help us quantify the error and quality of the model's predictions. Among the most common are:

1. **Mean Squared Error (MSE)**: This metric calculates the average of the squared errors, that is, the difference between the predicted values $\hat{y}_i$ and the actual values y_i. MSE heavily penalizes large errors due to squaring, making it a metric that is sensitive to outliers. A lower MSE indicates a better fit of the model to the data. The MSE formula is:

   $$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

   where n is the number of observations.

2. **R-squared (R^2)**: Also known as the coefficient of determination, this metric indicates the proportion of the variance in the dependent variable that is explained by the model's independent variables. For a nonconstant target, R^2 has a maximum of 1 and can be negative. A value of 1 indicates perfect predictions, while 0 corresponds to predicting the mean of the target values in the evaluation set. A negative value means that the predictions are worse than that mean-based reference. This reference is distinct from the training-mean baseline evaluated below. The R^2 formula is:

   $$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

   where $\bar{y}$ is the mean of the actual values y_i.

3. **Mean Absolute Error (MAE)**: This metric calculates the average of the absolute errors, that is, the absolute difference between the predicted values $\hat{y}_i$ and the actual values y_i. Unlike MSE, MAE does not penalize large errors as much, making it more robust to outliers. A lower MAE indicates better model performance. The MAE formula is:

   $$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

   where n is the number of observations.

These metrics provide different perspectives on model performance and, when used together, can provide a more complete and robust evaluation of the quality of the linear regression model's predictions.

In [16]:
```python
# Make predictions on the test set
y_pred = regression_model.predict(X_test)

# Import the metrics to evaluate the model
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Calculate the evaluation metrics
mse = mean_squared_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)

# Print the evaluation metrics
print(f'Mean Squared Error (MSE): {mse}')
print(f'Coefficient of Determination (R^2): {r2}')
print(f'Mean Absolute Error (MAE): {mae}')
print(f'Root Mean Squared Error (RMSE): {rmse}')
```

```
Mean Squared Error (MSE): 33596915.85136146
Coefficient of Determination (R^2): 0.7835929767120723
Mean Absolute Error (MAE): 4181.194473753641
Root Mean Squared Error (RMSE): 5796.2846592762735
```

Compare against a mean-prediction baseline

A baseline predicts the mean medical charge from the training set for every test case. Compare both models on the same test set: lower MAE and RMSE indicate smaller errors, measured in dollars. The training mean is used to avoid learning from the test targets.

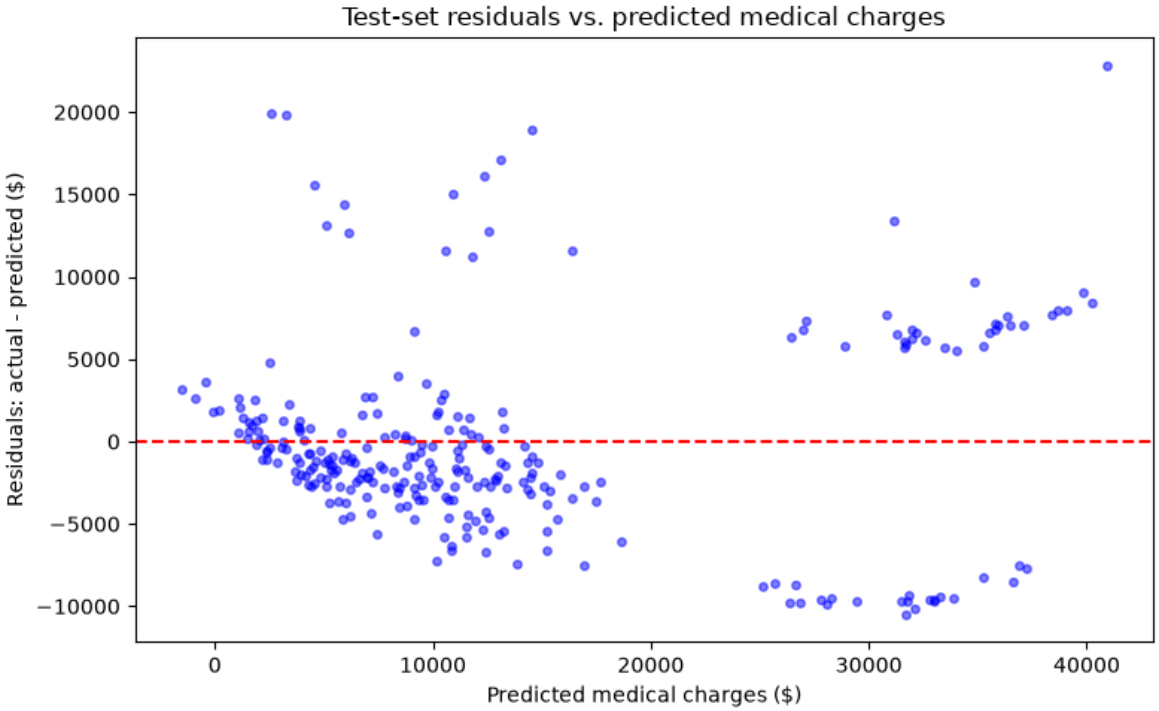
In [17]:
```
from sklearn.dummy import DummyRegressor

baseline_model = DummyRegressor(strategy='mean')
baseline_model.fit(X_train, y_train)
y_pred_baseline = baseline_model.predict(X_test)

comparison = pd.DataFrame({
    'Model': ['Training mean baseline', 'Linear regression'],
    'MAE ($)': [mean_absolute_error(y_test, y_pred_baseline), mae],
    'RMSE ($)': [np.sqrt(mean_squared_error(y_test, y_pred_baseline)), rmse],
    'R²': [r2_score(y_test, y_pred_baseline), r2]
})
print(comparison.to_string(index=False))
```

```
                Model     MAE ($)     RMSE ($)        R²
Training mean baseline 9593.338461 12465.610442 -0.000919
     Linear regression 4181.194474  5796.284659  0.783593
```

Inspect the residuals

A residual is the actual charge minus the predicted charge. Positive residuals indicate underestimation; negative residuals indicate overestimation. Look for residuals scattered around zero. Curves or distinct groups may suggest relationships the model misses, while a widening spread may indicate that error variability changes with predicted charges.

In [18]:
```
residuals = y_test - y_pred

plt.figure(figsize=(8, 5))
plt.scatter(y_pred, residuals, color='blue', alpha=0.5, s=15)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('Predicted medical charges ($)')
plt.ylabel('Residuals: actual - predicted ($)')
plt.title('Test-set residuals vs. predicted medical charges')
plt.tight_layout()
plt.show()
```



Make predictions for a new insurance company customer

In [ ]:
```
# Define a new test case with the following values
new_case = pd.DataFrame({
    'age': [30],  # Age
    'sex': [1],  # Gender (0: female, 1: male)
    'bmi': [19],  # Body mass index
    'children': [1],  # Number of children
    'smoker': [0],  # Smoker (0: no, 1: yes)
    'region_northeast': [0],  # Northeast region
    'region_northwest': [1],  # Northwest region
    'region_southeast': [0],  # Southeast region
    'region_southwest': [0]  # Southwest region
})

# Make the prediction with the new test case
prediction = regression_model.predict(new_case)

# Print the prediction result
print(f'The estimated cost of medical insurance is: ${prediction[0]:.2f}')
```

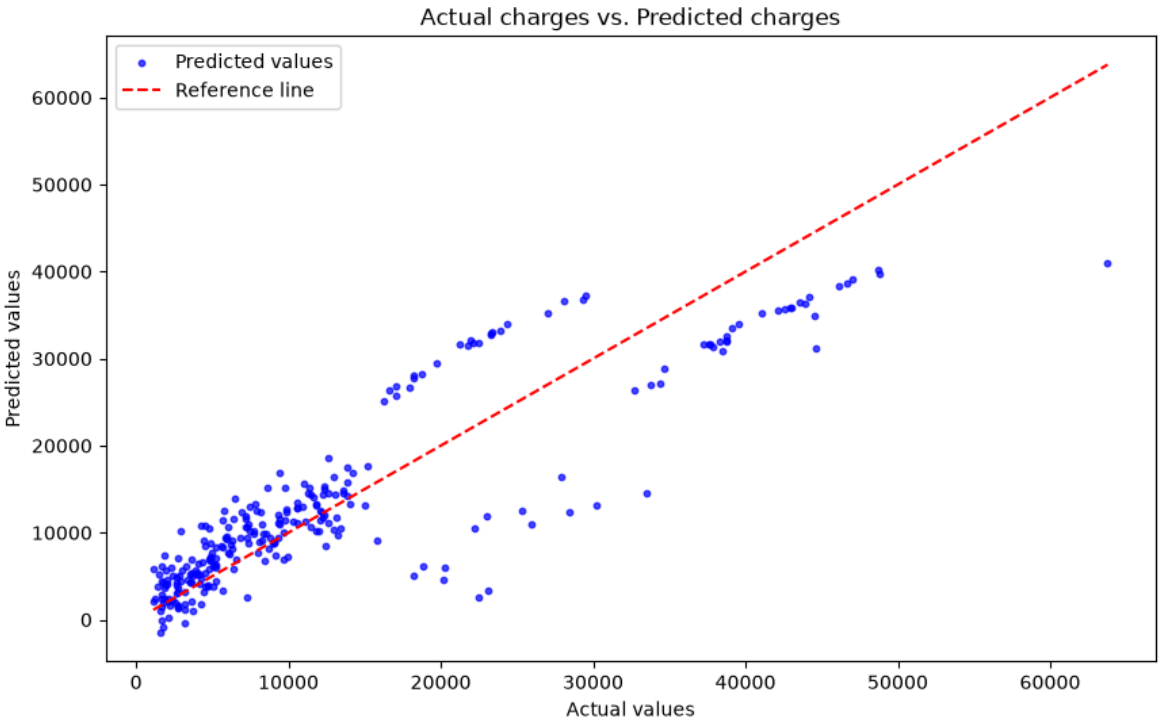
```
The estimated cost of medical insurance is: $2218.82
```

In the case of multiple regression, where you have more than one independent variable, the relationship between the independent variables and the dependent variable is represented in a **higher-dimensional space**. Therefore, instead of a regression line (which applies to simple regression with a single independent variable), you would have a hyperplane in a multidimensional space.

However, you can visualize the relationship between the actual values and the predicted values in a scatter plot, as shown below. This type of plot is useful for evaluating the accuracy of the model's predictions.

In [20]:
```
# Plot the actual values vs the predicted values
plt.figure(figsize=(10, 6))
```

```
plt.scatter(y_test, y_pred, color='blue', alpha=0.7, s=10)
plt.title('Actual charges vs. Predicted charges')
plt.xlabel('Actual values')
plt.ylabel('Predicted values')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', li
plt.legend(['Predicted values', 'Reference line'])
plt.show()
```



In this plot:

- The blue points represent the actual values against the predicted values.
- The dashed red line is a reference line that indicates where the points would be if the predictions were perfect (that is, $y_{\text{actual}} = y_{\text{predicted}}$).

This plot allows you to visualize how close the predictions are to the actual values. If the predictions are perfect, all the points will fall on the reference line.

Note: Negative predicted values can occur because ordinary linear regression does not constrain its predictions. Negative medical charges have no practical meaning.

Possible approaches include:

1. **Target transformation**: Consider fitting the model to `log1p(charges)` and converting predictions back with `expm1`. Evaluate performance in dollars after the inverse transformation; this changes the fitting objective and may introduce retransformation bias. Standardizing the predictors alone does not prevent negative predictions.

2. **Models with an appropriate output range**: Consider a model designed for positive targets, such as a Gamma regressor with a log link when all charges are strictly positive. `LinearRegression(positive=True)` only constrains the coefficients to be nonnegative; it does not guarantee nonnegative predictions.

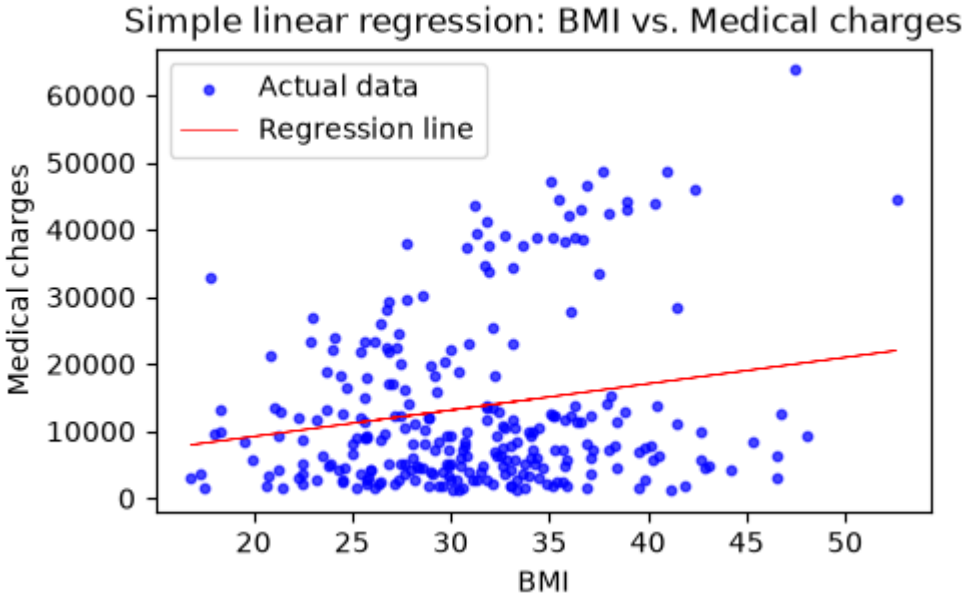
3. **Post-processing predictions**: If the application requires it, clip negative predictions to zero and evaluate the resulting predictions explicitly.

Simple Linear Regression

As an example, a feature can be fitted with simple linear regression. This will allow you to visualize the relationship between a single independent variable and the dependent variable.

We will fit a simple linear regression using one feature, for example, `bmi`.

In [21]:
```
# Copy the 'insurance.csv' dataset into a new DataFrame
df = insurance.copy()

X_2 = df[['bmi']]
y_2 = df['charges']

# Split the data into training and test sets
X_train2, X_test2, y_train2, y_test2 = train_test_split(X_2, y_2, test_size=0.2,

# Create and train the linear regression model
model = LinearRegression()
model.fit(X_train2, y_train2)

# Make predictions
y_pred = model.predict(X_test2)

# Plot the data and the regression line
plt.figure(figsize=(5, 3))
plt.scatter(X_test2, y_test2, color='blue', alpha=0.7, s=10, label='Actual data'
plt.plot(X_test2, y_pred, color='red', linewidth=0.5, label='Regression line')
plt.title('Simple linear regression: BMI vs. Medical charges')
plt.xlabel('BMI')
plt.ylabel('Medical charges')
plt.legend()
plt.show()
```



In [22]:
```
# Extract the linear regression coefficients
coefficients = model.coef_
intercept = model.intercept_

# Print the regression line equation
print(f'Regression line equation: y = {coefficients[0]:.2f}X + {intercept:.2f}')
```

```
Regression line equation: y = 392.44X + 1353.07
```

In this example:

1. **Data**: We use the `bmi` feature and the `charges` target variable.
2. **Data splitting**: We split the data into training and test sets.
3. **Model**: We create and train a simple linear regression model.
4. **Predictions**: We make predictions on the test set.
5. **Plot**: We plot the actual data and the fitted regression line.

This plot will allow you to visualize how BMI relates to medical charges and how the fitted regression line represents this relationship.

Dataset standardization

In [23]:
```
# Standardize the data in the 'insurance.csv' dataset with the StandardScaler fu
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# Apply standardization to the training and test sets
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [24]:
```
# Convert the standardized data to DataFrames to make interpretation easier
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X.columns)

# Display the first rows of the standardized data
X_train_scaled_df.head()
```

Out[24]:

	age	sex	bmi	children	smoker	region_northeast	region_northv
0	0.472227	-1.024602	-1.756525	0.734336	-0.508747	-0.576631	1.783
1	0.543313	-1.024602	-1.033082	-0.911192	-0.508747	1.734212	-0.560
2	0.898745	-1.024602	-0.943687	-0.911192	-0.508747	-0.576631	-0.560
3	-0.025379	-1.024602	0.622393	3.202629	-0.508747	-0.576631	-0.560
4	1.040918	-1.024602	-1.504893	1.557100	-0.508747	-0.576631	1.783

In [25]:
```
# Create an instance of the linear regression model
scaled_regression_model = LinearRegression()

# Fit the linear regression model with the standardized data
scaled_regression_model.fit(X_train_scaled_df, y_train)
```

Out[25]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

In [26]:
```
# Make predictions on the test set
y_pred_scaled = scaled_regression_model.predict(X_test_scaled_df)

# Calculate the evaluation metrics
mse_scaled = mean_squared_error(y_test, y_pred_scaled)
r2_scaled = r2_score(y_test, y_pred_scaled)
mae_scaled = mean_absolute_error(y_test, y_pred_scaled)
rmse_scaled = np.sqrt(mse_scaled)

# Print the evaluation metrics
print(f'Mean Squared Error (MSE) - Scaled Data: {mse_scaled}')
print(f'Coefficient of Determination (R^2) - Scaled Data: {r2_scaled}')
print(f'Mean Absolute Error (MAE) - Scaled Data: {mae_scaled}')
print(f'Root Mean Squared Error (RMSE) - Scaled Data: {rmse_scaled}')
```

```
Mean Squared Error (MSE) - Scaled Data: 33596915.851361476
Coefficient of Determination (R^2) - Scaled Data: 0.7835929767120722
Mean Absolute Error (MAE) - Scaled Data: 4181.194473753652
Root Mean Squared Error (RMSE) - Scaled Data: 5796.284659276274
```

For ordinary least squares with an intercept, standardizing the predictors preserves the fitted predictions up to numerical precision. StandardScaler changes the scale of `X`, and the model adjusts its coefficients and intercept to compensate.

Scaling allows coefficients to be compared for a one-standard-deviation increase in each predictor. However, these magnitudes do not by themselves establish predictive importance or causality, and correlated predictors can affect their interpretation. For binary indicators, the scaled coefficient no longer represents a direct change from 0 to 1; use the unscaled coefficient for that interpretation.

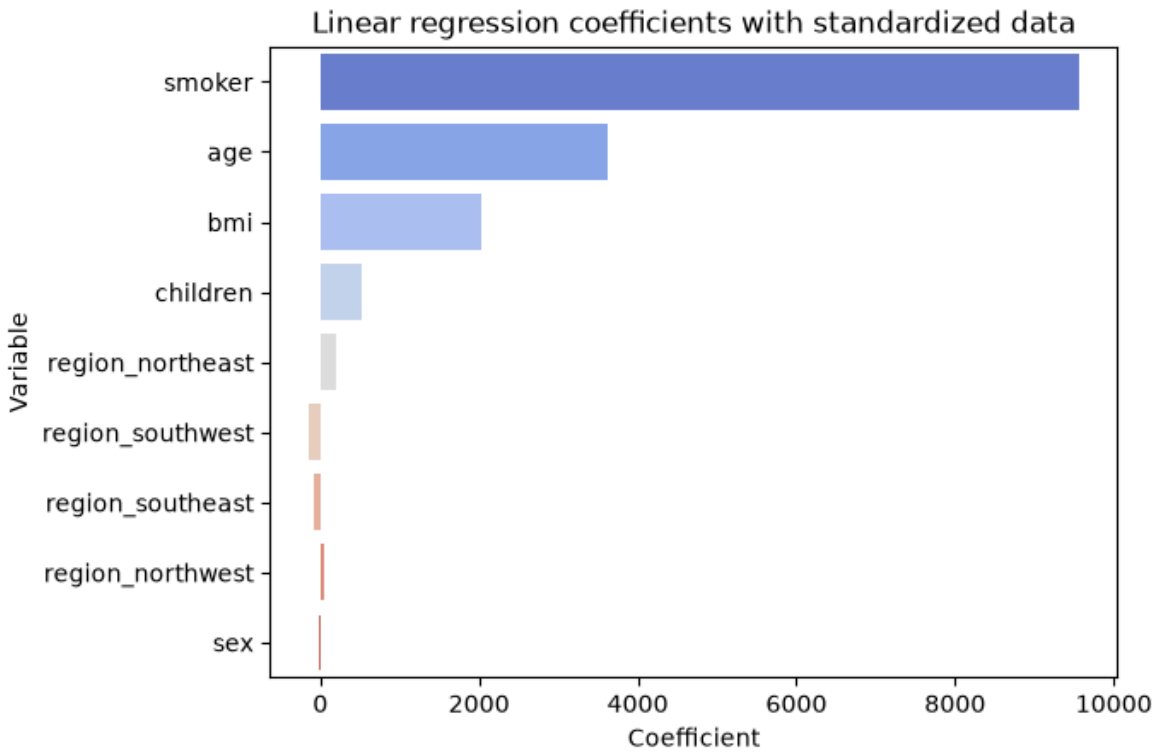
With all four region indicators and an intercept, coefficients are not unique. Refitting after scaling can select a different allocation among the redundant region columns, even when predictions agree. For the coefficient-to-SHAP comparison below, use the same original fitted model and the same training-data background throughout.

In [27]:
```
# Plot coefficient magnitudes for the linear regression model with standardized
scaled_coefficients = scaled_regression_model.coef_
scaled_coef_df = pd.DataFrame({'Variable': X.columns, 'Coefficient': scaled_coef
scaled_coef_df['Abs_Coefficient'] = scaled_coef_df['Coefficient'].abs()
scaled_coef_df = scaled_coef_df.sort_values(by='Abs_Coefficient', ascending=Fals
print(scaled_coef_df)

# Plot the coefficients sorted from highest to lowest in a bar chart
sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwar
plt.xlabel('Coefficient')
plt.ylabel('Variable')
```

```
plt.title('Linear regression coefficients with standardized data')
plt.show()
```

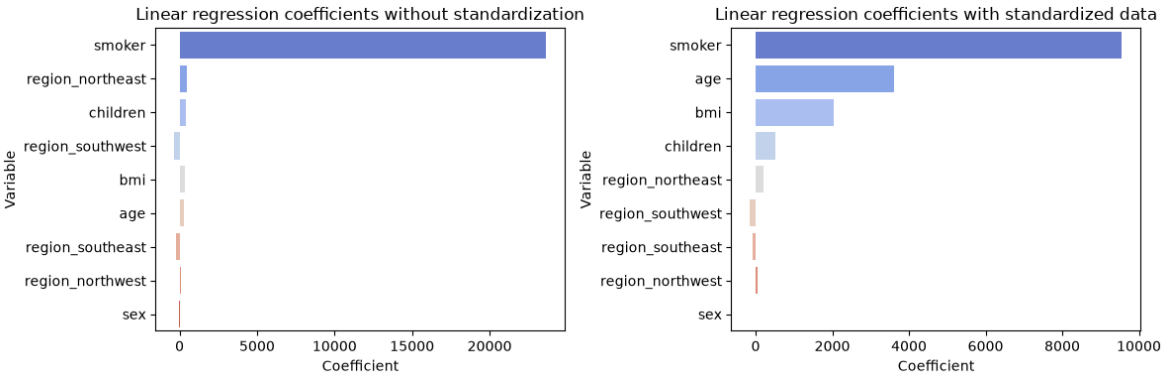
	Variable	Coefficient	Abs_Coefficient
4	smoker	9558.481409	9558.481409
0	age	3614.975415	3614.975415
2	bmi	2036.228123	2036.228123
3	children	516.890247	516.890247
5	region_northeast	199.698058	199.698058
8	region_southwest	-150.167101	150.167101
7	region_southeast	-86.621147	86.621147
6	region_northwest	38.734949	38.734949
1	sex	-9.293101	9.293101

```
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_28508\3480408063.py:9: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwa
rm')
```



In [28]:
```
# Plot the linear regression coefficients with unstandardized data (coef_df) and
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')
plt.xlabel('Coefficient')
plt.ylabel('Variable')
plt.title('Linear regression coefficients without standardization')
plt.subplot(1, 2, 2)
sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwar
plt.xlabel('Coefficient')
plt.ylabel('Variable')
plt.title('Linear regression coefficients with standardized data')
plt.tight_layout()
plt.show()
```

```
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_28508\956020363.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_28508\956020363.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwa
rm')
```



Assignment

1. **Find a new dataset:**
   - **Find** a new dataset and **identify** the predictor and target variable(s).

Descripción:

El conjunto de datos de rendimiento estudiantil es un conjunto de datos diseñado para examinar los factores que influyen en el rendimiento académico de los estudiantes. El conjunto de datos consta de 10.000 registros de estudiantes, con cada registro que contiene información sobre varios predictores y un índice de rendimiento. Variables:

```
Horas estudiadas: El número total de horas dedicadas a estudiar
por cada alumno.
Puntuaciones anteriores : Las puntuaciones obtenidas por los
estudiantes en exámenes anteriores.
Actividades extracurriculares: Si el estudiante participa en
actividades extracurriculares (Sí o No).
Horas de sueño: El número promedio de horas de sueño que el
estudiante tenía por día.
Ejemplos de documentos de preguntas practicados : El número de
documentos de preguntas de muestra que el estudiante practicó.
```

Variable objetivo :

```
Índice de rendimiento: una medida del rendimiento general de
cada estudiante. El índice de rendimiento representa el
rendimiento académico del estudiante y se ha redondeado al
número entero más cercano. El índice oscila entre 10 y 100, con
valores más altos que indican un mejor rendimiento.
```

El conjunto de datos tiene como objetivo proporcionar información sobre la relación entre las variables predictoras y el índice de rendimiento. Los investigadores y analistas de datos pueden usar este conjunto de datos para explorar el impacto de estudiar horas, puntajes anteriores, actividades extracurriculares, horas de sueño y documentos de preguntas de muestra sobre el rendimiento de los estudiantes.

P.D.: Tenga en cuenta que este conjunto de datos es sintético y creado con fines ilustrativos. Las relaciones entre las variables y el índice de rendimiento pueden no reflejar escenarios del mundo real

In [1]:
```
# Write your code here..
# Source data: https://www.kaggle.com/datasets/nikhil7280/student-performance-mu

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import numpy as np
from sklearn.linear_model import LinearRegression # Linear regression with sciki

df=pd.read_csv("student_performance.csv")
print(df.shape)
df.head()
```

```
(10000, 6)
```

Out[1]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
0	7	99	Yes	9	1	91.0
1	4	82	No	4	2	65.0
2	8	51	Yes	7	2	45.0
3	5	52	Yes	5	2	36.0
4	7	75	No	8	5	66.0

In [2]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column                            Non-Null Count  Dtype
---  ------                            --------------  -----
 0   Hours Studied                     10000 non-null  int64
 1   Previous Scores                   10000 non-null  int64
 2   Extracurricular Activities        10000 non-null  str
 3   Sleep Hours                       10000 non-null  int64
 4   Sample Question Papers Practiced  10000 non-null  int64
 5   Performance Index                 10000 non-null  float64
dtypes: float64(1), int64(4), str(1)
memory usage: 468.9 KB
```

In [3]:
```
df.describe().T
```

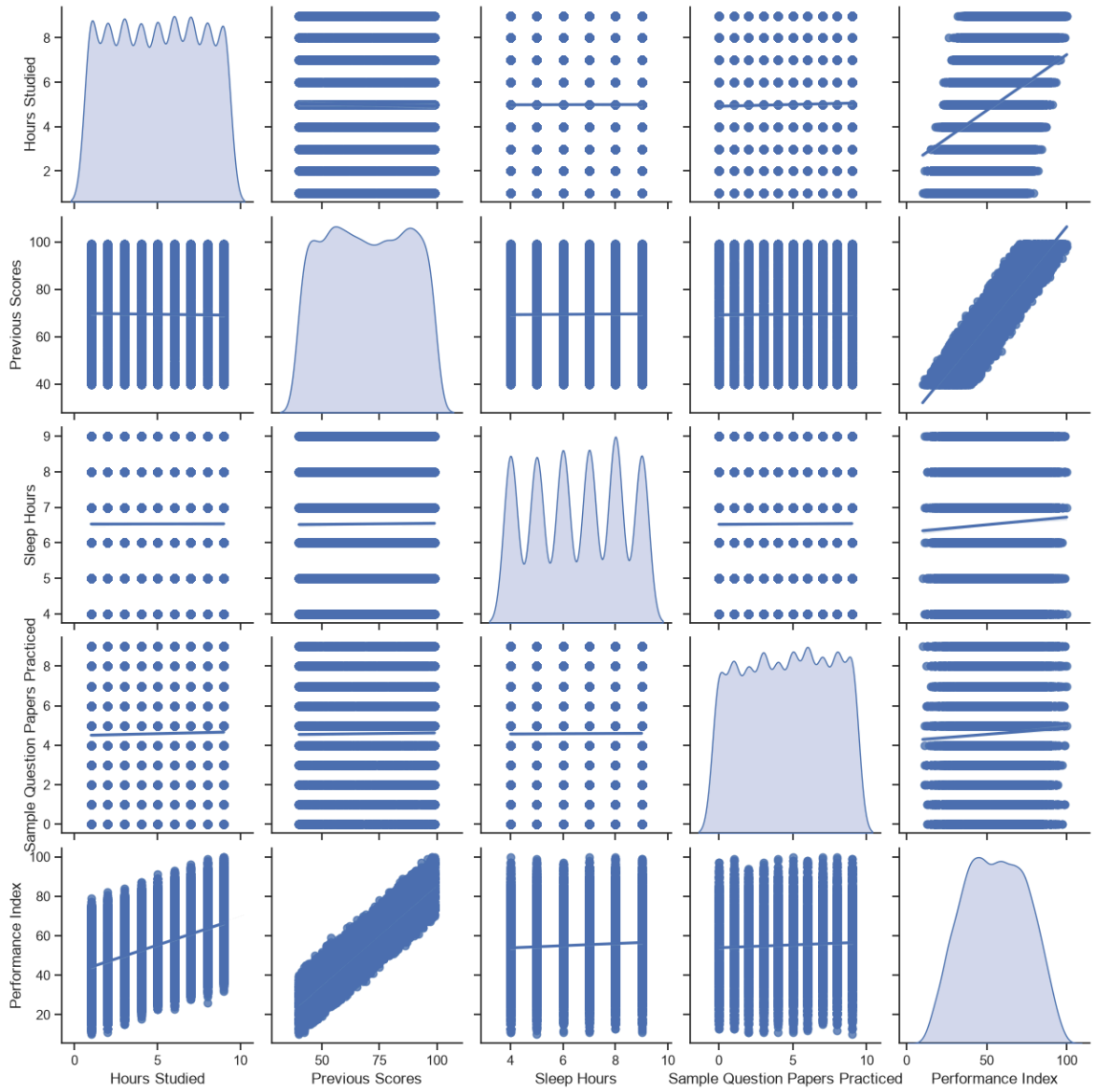
Out[3]:

	count	mean	std	min	25%	50%	75%	max
Hours Studied	10000.0	4.9929	2.589309	1.0	3.0	5.0	7.0	9.0
Previous Scores	10000.0	69.4457	17.343152	40.0	54.0	69.0	85.0	99.0
Sleep Hours	10000.0	6.5306	1.695863	4.0	5.0	7.0	8.0	9.0
Sample Question Papers Practiced	10000.0	4.5833	2.867348	0.0	2.0	5.0	7.0	9.0
Performance Index	10000.0	55.2248	19.212558	10.0	40.0	55.0	71.0	100.0

In [4]:
```
df["Extracurricular Activities"].unique()
```

Out[4]:
```
<StringArray>
['Yes', 'No']
Length: 2, dtype: str
```

In [5]:
```
# Configurar el estilo visual (opcional pero recomendado)
sns.set_theme(style="ticks")

# Crear el pairplot con una línea de regresión para cada gráfico de dispersión
sns.pairplot(df, kind="reg", diag_kind="kde")

# Mostrar la gráfica
plt.show()
```



In [6]:
```
# Encode the categorical variable
df['Extracurricular Activities'] = df['Extracurricular Activities'].replace({'No
```

In [7]:
```
# Select the independent variables (X)
X = df.drop('Performance Index', axis=1)
X
```

Out[7]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced
0	7	99	1	9	1
1	4	82	0	4	2
2	8	51	1	7	2
3	5	52	1	5	2
4	7	75	0	8	5
...	...	...	...	...	...
9995	1	49	1	4	2
9996	7	64	1	8	5
9997	6	83	1	8	5
9998	9	97	1	7	0
9999	7	74	0	8	1

10000 rows × 5 columns

In [9]:
```
# Select the dependent variable (y)
y = df['Performance Index']
y
```

Out[9]:
```
0       91.0
1       65.0
2       45.0
3       36.0
4       66.0
        ...
9995    23.0
9996    58.0
9997    74.0
9998    95.0
9999    64.0
Name: Performance Index, Length: 10000, dtype: float64
```

2. **Model Fitting:**
   - **Fit** a linear regression model.
   - **Evaluate** the model's performance (e.g., `RMSE`) based on its unit of measurement.
   - **Analyze** the most significant coefficients for the model.
   - (Optional) Use `cross_val_score` with 5 folds. Calculate the mean RMSE and the standard deviation of all the scores obtained with cross-validation (optional).

In [10]:
```
# Write your code here..
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

In [11]:
```
# Create an instance of the linear regression model
regression_model1 = LinearRegression()

# Fit the model to the training data
regression_model1.fit(X_train, y_train)
```

Out[11]:

▾ LinearRegression

▸ Parameters

▸ Fitted attributes

In [12]:
```
# Make predictions on the test set
y_pred = regression_model1.predict(X_test)
```

In [13]:
```
# Import the metrics to evaluate the model
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Calculate the evaluation metrics
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)

# Print the evaluation metrics
print(f'Mean Squared Error (MSE): {mse}')
print(f'Coefficient of Determination (R^2): {r2}')
print(f'Mean Absolute Error (MAE): {mae}')
print(f'Root Mean Squared Error (RMSE): {rmse}')
```

```
Mean Squared Error (MSE): 4.082628398521853
Coefficient of Determination (R^2): 0.9889832909573145
Mean Absolute Error (MAE): 1.6111213463123044
Root Mean Squared Error (RMSE): 2.0205515085050054
```

El RMSE nos indica que el modelo de regresion lineal multiple para determinar el valor Performance Index puede variar +/- 2.02 puntos

In [14]:
```
# Get the coefficients
coefficients = regression_model1.coef_
coefficients
```

Out[14]:
```
array([2.85248393, 1.0169882 , 0.60861668, 0.47694148, 0.19183144])
```

In [15]:
```
# Get the intercept
intercept = regression_model1.intercept_
intercept
```

Out[15]:
```
np.float64(-33.921946215556126)
```

In [16]:
```
# Define the regression line equation
line_equation = 'y = ' + ' + '.join([f'{coefficients[i]}*X{i}' for i in range(le
print(f'Regression line equation: {line_equation}')
```

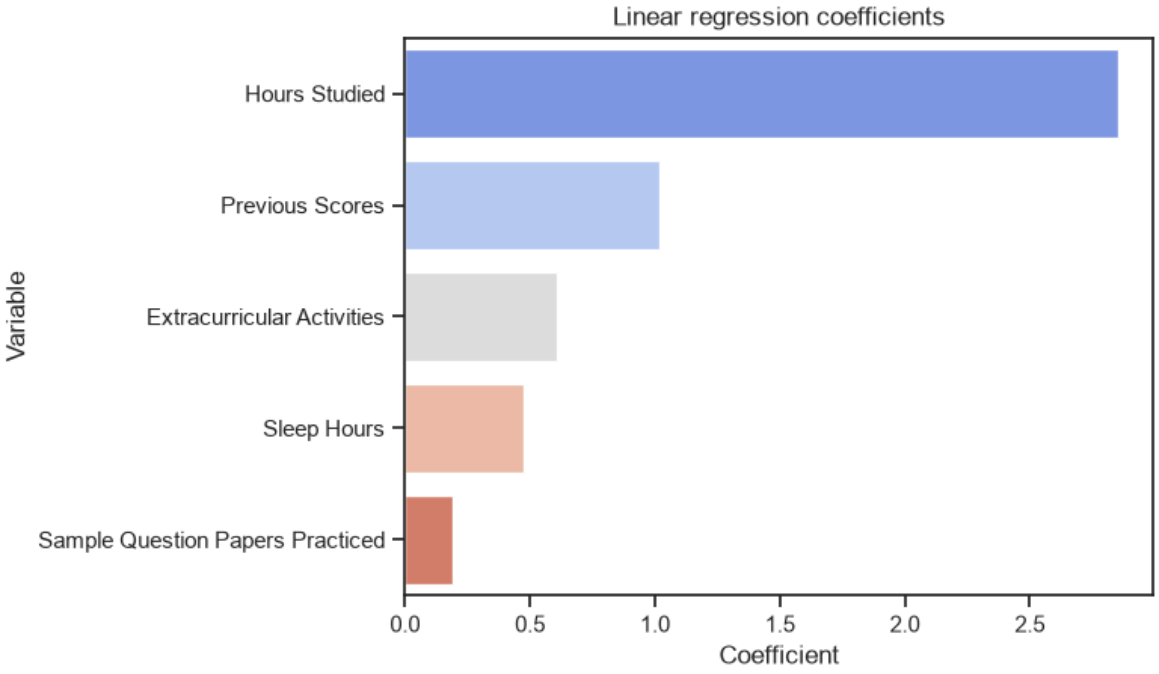
```
Regression line equation: y = 2.852483930072525*X0 + 1.016988198932932*X1 + 0.608
6166795764233*X2 + 0.47694148417627186*X3 + 0.19183144145054268*X4 + -33.92194621
5556126
```

In [17]:
```python
# Create a DataFrame to make interpretation easier
coef_df = pd.DataFrame({'Variable': X.columns, 'Coefficient': coefficients})

# Sort by absolute coefficient magnitude; the predictors have different units
coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()

# Sort the coefficients from highest to lowest
coef_df = coef_df.sort_values(by='Abs_Coefficient', ascending=False)

print(coef_df)
```

```
                          Variable  Coefficient  Abs_Coefficient
0                    Hours Studied     2.852484         2.852484
1                 Previous Scores      1.016988         1.016988
2        Extracurricular Activities     0.608617         0.608617
3                      Sleep Hours      0.476941         0.476941
4  Sample Question Papers Practiced     0.191831         0.191831
```

In [18]:
```python
# Plot the coefficients sorted from highest to lowest in a bar chart
sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')

plt.xlabel('Coefficient')
plt.ylabel('Variable')
plt.title('Linear regression coefficients')
plt.show()
```

```
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_16012\789844377.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')
```



La variable mas significativa en la determinacion del Performance Index es Hour Studied con 2.85 y le sigue Previous Scores con un valor de 1.01.

In [20]:
```python
from sklearn.model_selection import cross_val_score

# 1. Calcular los scores usando 'neg_root_mean_squared_error'
# Se recomienda usar X_train e y_train para evitar fugas de datos (data leakage)
scores = cross_val_score(
    regression_model1,
    X_train,
    y_train,
    cv=5,
    scoring='neg_root_mean_squared_error'
)

# 2. Convertir a valores positivos de RMSE
rmse_scores = -scores

# 3. Calcular la media y la desviación estándar
mean_rmse = rmse_scores.mean()
std_rmse = rmse_scores.std()

# Mostrar resultados
print(f"RMSE de cada uno de los 5 folds: {rmse_scores}\n")
print(f"Mean RMSE: {mean_rmse:.4f}")
print(f"Standard Deviation: {std_rmse:.4f}")
```

```
RMSE de cada uno de los 5 folds: [2.04474939 2.07424914 2.01798298 2.06766808 2.0
1587601]

Mean RMSE: 2.0441
Standard Deviation: 0.0243
```

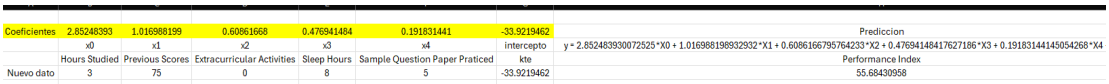
El RSME promedio calculado a partir de una validacion cruzada de 5 folds se determino que el valor promedio de ajuste es de 2.0441 puntos , valor que es proximo al calculado en test con 2.0205 puntos.

4. **Predicting New Records:**
   - Use the fitted model to **predict** the value of `y` for the previous record in an Excel spreadsheet. Paste a screenshot of the prediction here:

In [21]:
```python
new_case = pd.DataFrame({
    'Hours Studied': [3],
    'Previous Scores': [75],
    'Extracurricular Activities': [0],
    'Sleep Hours': [8],
    'Sample Question Papers Practiced': [5]

})

# Make the prediction with the new test case
prediction = regression_model1.predict(new_case)

# Print the prediction result
print(f'The estimated cost of medical insurance is: ${prediction[0]:.2f}')
```

```
The estimated cost of medical insurance is: $55.68
```

Paste the screenshot here..



Coeficientes	2.85248393	1.016988199	0.60861668	0.476941484	0.191831441	-33.9219462	Prediccion
	x0	x1	x2	x3	x4	intercepto	y = 2.852483930072525*X0 + 1.016988198932932*X1 + 0.6086166795764233*X2 + 0.4769414841762718 6*X3 + 0.19183144145054268*X4
	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Paper Praticed	kte	Performance Index
Nuevo dato	3	75	0	8	5	-33.9219462	55.68430958

5. **Possible improvements:**
   - Suggest an improvement proposal to optimize **model performance**.
   - Agregar un proceso de escalado para comparar diferentes caracterisitcas que se encuentran en diferentes medidas, esto agregar mayor interpretabilidad a la comparacion de caracteristicas relevantes

In [22]:
```
# Write your code here..
# Realizar un proceso de escalado de los datos en el conjunto de datos 'student_
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# Apply standardization to the training and test sets
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert the standardized data to DataFrames to make interpretation easier
X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X.columns)

# Display the first rows of the standardized data
X_train_scaled_df.head()
```

Out[22]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced
0	0.006455	-1.183844	-0.989307	0.269888	0.134041
1	-1.149747	-1.241506	1.010808	0.269888	0.483562
2	-1.149747	0.661318	-0.989307	0.269888	-0.914520
3	-1.149747	-1.356828	-0.989307	-0.318906	-1.264041
4	1.162658	-1.299167	-0.989307	1.447477	-1.613561

In [23]:
```
# Create an instance of the linear regression model
scaled_regression_model = LinearRegression()

# Fit the linear regression model with the standardized data
scaled_regression_model.fit(X_train_scaled_df, y_train)

# Make predictions on the test set
y_pred_scaled = scaled_regression_model.predict(X_test_scaled_df)

# Calculate the evaluation metrics
mse_scaled = mean_squared_error(y_test, y_pred_scaled)
r2_scaled = r2_score(y_test, y_pred_scaled)
mae_scaled = mean_absolute_error(y_test, y_pred_scaled)
rmse_scaled = np.sqrt(mse_scaled)

# Print the evaluation metrics
print(f'Mean Squared Error (MSE) - Scaled Data: {mse_scaled}')
```

```
print(f'Coefficient of Determination (R^2) - Scaled Data: {r2_scaled}')
print(f'Mean Absolute Error (MAE) - Scaled Data: {mae_scaled}')
print(f'Root Mean Squared Error (RMSE) - Scaled Data: {rmse_scaled}')
```

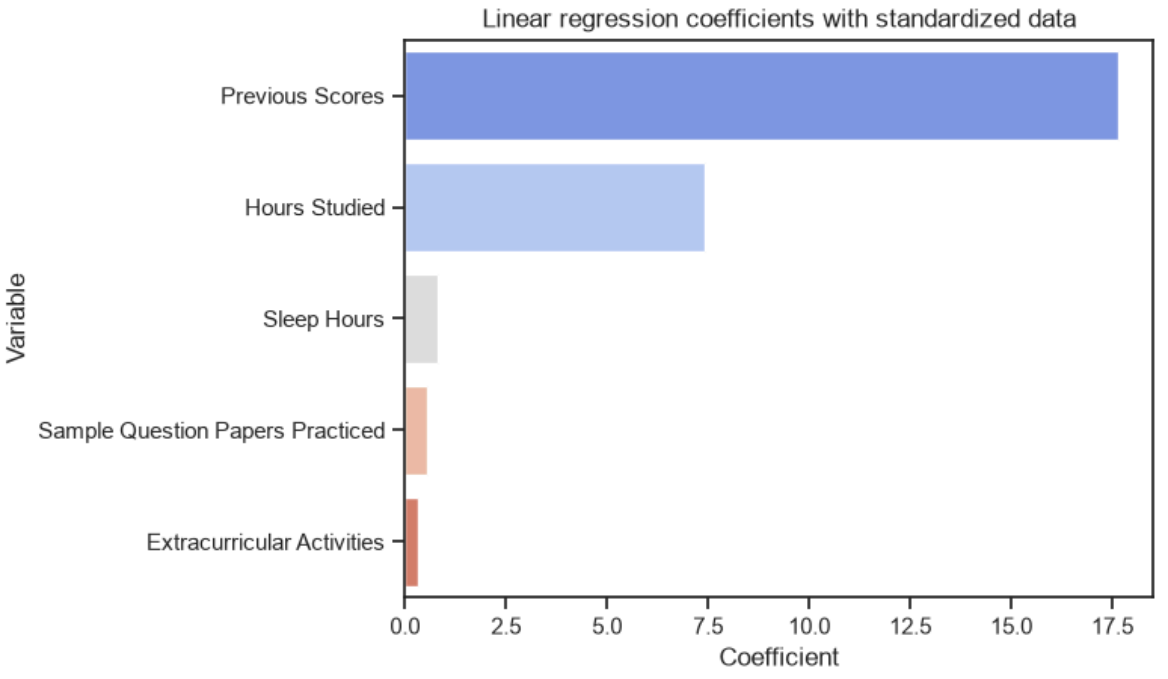
```
Mean Squared Error (MSE) - Scaled Data: 4.082628398521842
Coefficient of Determination (R^2) - Scaled Data: 0.9889832909573145
Mean Absolute Error (MAE) - Scaled Data: 1.6111213463123022
Root Mean Squared Error (RMSE) - Scaled Data: 2.0205515085050028
```

In [24]:
```
# Plot coefficient magnitudes for the linear regression model with standardized
scaled_coefficients = scaled_regression_model.coef_
scaled_coef_df = pd.DataFrame({'Variable': X.columns, 'Coefficient': scaled_coef
scaled_coef_df['Abs_Coefficient'] = scaled_coef_df['Coefficient'].abs()
scaled_coef_df = scaled_coef_df.sort_values(by='Abs_Coefficient', ascending=Fals
print(scaled_coef_df)
```

```
                          Variable  Coefficient  Abs_Coefficient
1                  Previous Scores    17.637271        17.637271
0                    Hours Studied     7.401341         7.401341
3                      Sleep Hours     0.810031         0.810031
4  Sample Question Papers Practiced     0.548842         0.548842
2         Extracurricular Activities     0.304291         0.304291
```

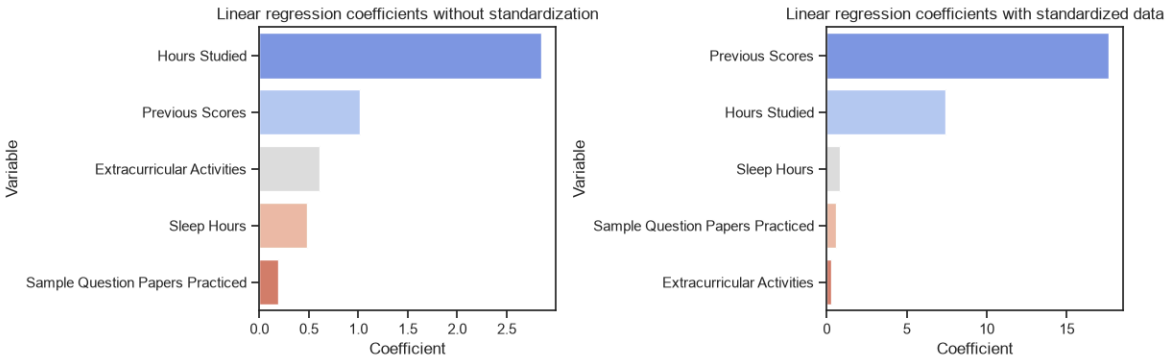
In [25]:
```
# Plot the coefficients sorted from highest to lowest in a bar chart
sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwar
plt.xlabel('Coefficient')
plt.ylabel('Variable')
plt.title('Linear regression coefficients with standardized data')
plt.show()
```

```
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_16012\822995905.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwa
rm')
```



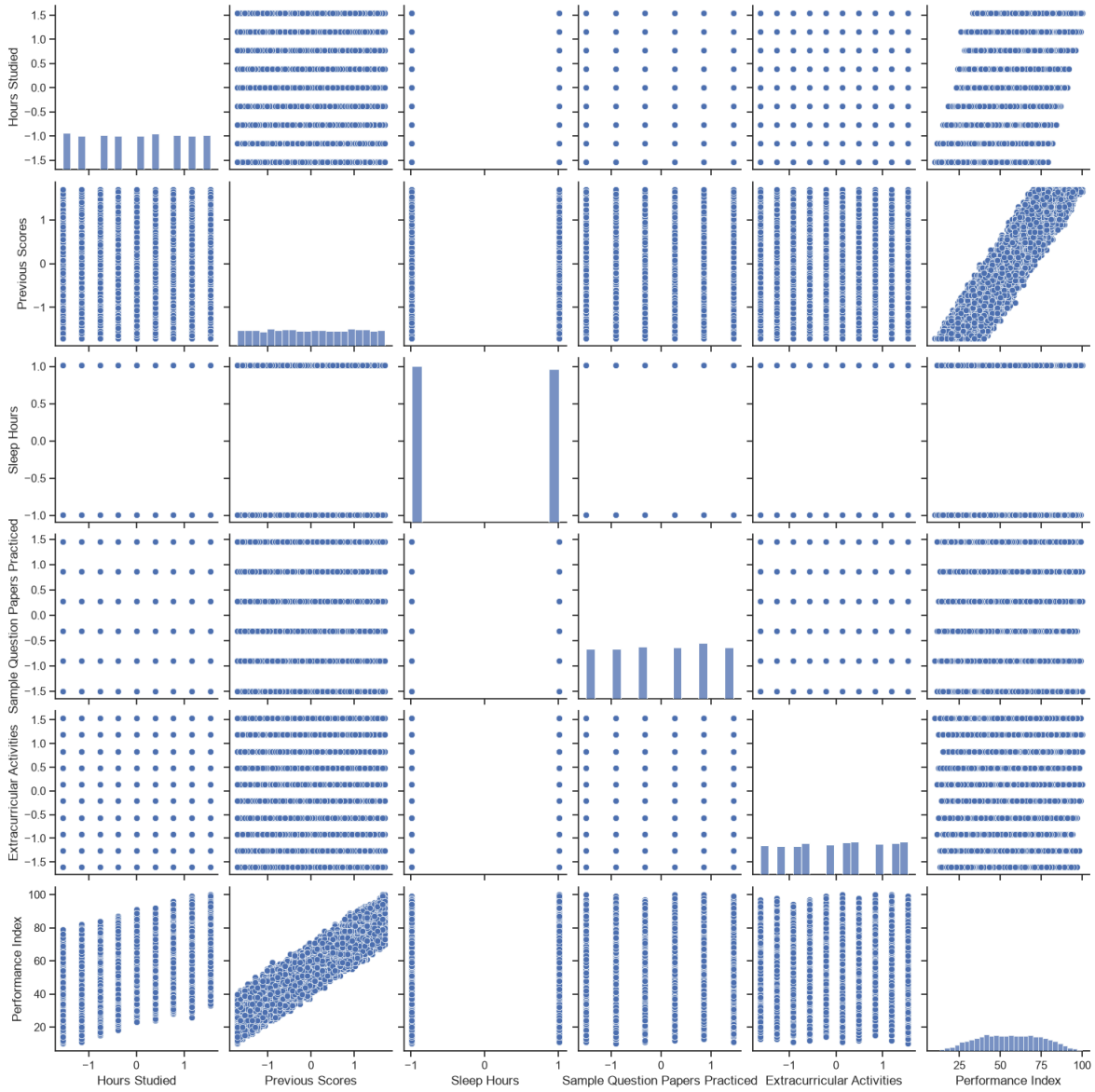
In [26]:
```
# Plot the linear regression coefficients with unstandardized data (coef_df) and
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')
plt.xlabel('Coefficient')
plt.ylabel('Variable')
plt.title('Linear regression coefficients without standardization')
plt.subplot(1, 2, 2)
sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwar
plt.xlabel('Coefficient')
plt.ylabel('Variable')
plt.title('Linear regression coefficients with standardized data')
plt.tight_layout()
plt.show()
```

```
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_16012\3488001194.py:4: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=coef_df, palette='coolwarm')
C:\Users\DiegoR\AppData\Local\Temp\ipykernel_16012\3488001194.py:9: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Coefficient', y='Variable', data=scaled_coef_df, palette='coolwa
rm')
```



In [33]:
```
# 1. Nombres de tus variables de entrada (X)
columnas_x = [
    "Hours Studied",
    "Previous Scores",
    "Sleep Hours",
    "Sample Question Papers Practiced",
    "Extracurricular Activities"
]

# 2. Crear el DataFrame con los datos estandarizados
df_total = pd.DataFrame(X_train_scaled, columns=columnas_x)

# 3. Extraer solo los valores numéricos de y_train para ignorar el índice antigu
# Usar .values garantiza que la fila 1 de X se junte correctamente con la fila 1
df_total["Performance Index"] = y_train.values
```

```
# 4. Generar el pairplot optimizado (con una muestra aleatoria de 1000 filas par
sns.pairplot(df_total)
plt.show()
```



Despues de aplicar el proceso de escalado de las caracteristicas se oberva que Previous Score se convierte en la variable dominante con un valor de coeficiente de 17.6, seguido de Hours Studied con 7.07 esto es importante ya que agrega comparacion entre variables ya que los cambios de puntuacion no sesgaran a una u otra variable; sin embargo, es importante mencionar que el Root Mean Squared Error (RMSE) - Scaled Data: 2.02 puntos esl mismo que no difiere sin el escalado con valor de RMSE 2.02 puntos, esto se debe a que la solucion geoemtrica en ambos casos es la misma pero en diferente sistema de referencia.

Conclusions:

Through this analysis, we have learned to use linear regression to predict the medical charges of an insurance company's customers. This technique allows us to better understand the relationships between different variables and make informed predictions that can guide medical insurance business strategies.

Grading Rubric

1. Dataset source and description of variables: **1pts**
2. Fitting the linear regressor (simple or multiple): **1pts**
3. Calculating the *RMSE* metric and identifying its unit of measurement: **1pts**
4. Predicting a **new example** using the `.predict` method in a Jupyter Notebook or `.py` script: **1pts**
5. Predicting the **new example** from the previous step using an Excel spreadsheet and comparing results: **3pts**
6. Proposing improvements to optimize the metrics: **3pts**

SHAP: summary and individual prediction

Run the model training cells first.

- **Beeswarm:** each dot is a customer. The horizontal axis shows contributions in dollars, not ages or numbers of children. Red = high feature value; blue = low. Left of zero lowers the prediction relative to the baseline; right raises it.
- **Waterfall:** change `person_index` to select a customer by position in `X_test`. Baseline + contributions = prediction. Red increases the prediction; blue decreases it.

In [33]:
```
%pip install shap
```

```
Requirement already satisfied: shap in c:\RD\2026_2027\M3V2\.venv\Lib\site-packag
es (0.54.1)
Requirement already satisfied: numpy>=2 in c:\RD\2026_2027\M3V2\.venv\Lib\site-pa
ckages (from shap) (2.5.4)
Requirement already satisfied: scipy in c:\RD\2026_2027\M3V2\.venv\Lib\site-packa
ges (from shap) (1.18.1)
Requirement already satisfied: scikit-learn in c:\RD\2026_2027\M3V2\.venv\Lib\sit
e-packages (from shap) (1.9.1)
Requirement already satisfied: pandas in c:\RD\2026_2027\M3V2\.venv\Lib\site-pack
ages (from shap) (3.0.6)
Requirement already satisfied: tqdm>=4.27.0 in c:\RD\2026_2027\M3V2\.venv\Lib\sit
e-packages (from shap) (4.70.1)
Requirement already satisfied: packaging>20.9 in c:\RD\2026_2027\M3V2\.venv\Lib\s
ite-packages (from shap) (26.3)
Requirement already satisfied: slicer==0.0.8 in c:\RD\2026_2027\M3V2\.venv\Lib\si
te-packages (from shap) (0.0.8)
Requirement already satisfied: cloudpickle in c:\RD\2026_2027\M3V2\.venv\Lib\site
-packages (from shap) (3.1.2)
Requirement already satisfied: colorama in c:\RD\2026_2027\M3V2\.venv\Lib\site-pa
ckages (from tqdm>=4.27.0->shap) (0.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\RD\2026_2027\M3V2\.ve
nv\Lib\site-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\RD\2026_2027\M3V2\.venv\Lib\site-pack
ages (from pandas->shap) (2026.5)
Requirement already satisfied: six>=1.5 in c:\RD\2026_2027\M3V2\.venv\Lib\site-pa
ckages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Requirement already satisfied: joblib>=1.4.0 in c:\RD\2026_2027\M3V2\.venv\Lib\si
te-packages (from scikit-learn->shap) (1.6.0)
Requirement already satisfied: narwhals>=2.0.1 in c:\RD\2026_2027\M3V2\.venv\Lib
\site-packages (from scikit-learn->shap) (2.27.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in c:\RD\2026_2027\M3V2\.venv
\Lib\site-packages (from scikit-learn->shap) (3.7.0)
Note: you may need to restart the kernel to use updated packages.
```

In [35]:

```python
#import shap

#explainer = shap.LinearExplainer(regression_model, X_train)
#shap_values = explainer(X_test)

#shap.plots.beeswarm(shap_values)

import shap

explainer = shap.LinearExplainer(regression_model, X_train)
shap_values = explainer(X_test)

# If shap_values is an Explanation object:
shap.plots.beeswarm(shap_values.values)

# or, if your version uses older API:
# shap.plots.beeswarm(shap_values)
```

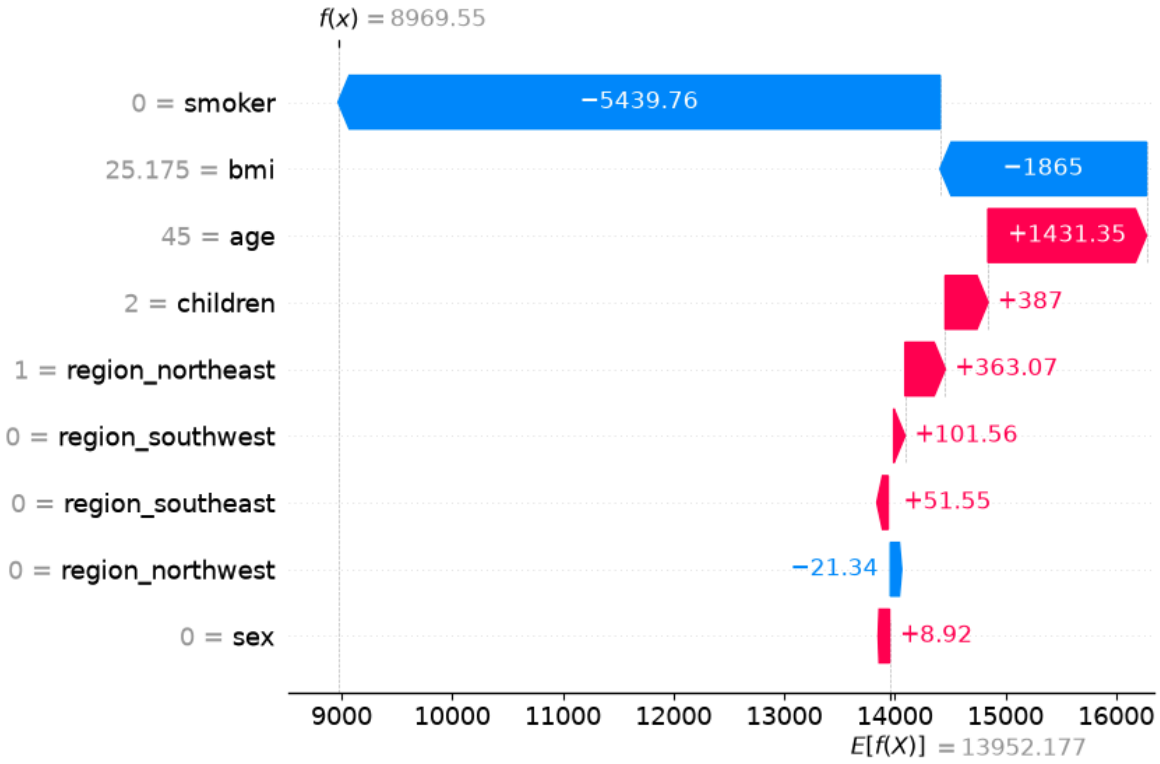
```
Background dataset has 1070 samples but max_samples=100. Subsampling to 100 sampl
es for SHAP value computation. To use all samples, set max_samples=1070 when init
ializing the masker.
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[35], line 14
     10 explainer = shap.LinearExplainer(regression_model, X_train)
     11 shap_values = explainer(X_test)
     12
     13 # If shap_values is an Explanation object:
---> 14 shap.plots.beeswarm(shap_values.values)
     15
     16 # or, if your version uses older API:
     17 # shap.plots.beeswarm(shap_values)

File c:\RD\2026_2027\M3V2\.venv\Lib\site-packages\shap\plots\_beeswarm.py:111, in beeswarm(shap_values, max_display, order, clustering, cluster_threshold, color, axis_color, alpha, ax, show, log_scale, color_bar, s, plot_size, color_bar_label, group_remaining_features)
    109 if not isinstance(shap_values, Explanation):
    110     emsg = "The beeswarm plot requires an `Explanation` object as the `shap_values` argument."
--> 111     raise TypeError(emsg)
    113 sv_shape = shap_values.shape
    114 if len(sv_shape) == 1:

TypeError: The beeswarm plot requires an `Explanation` object as the `shap_values` argument.
```

In [36]:
```
print(type(shap_values))
```

```
<class 'shap._explanation.Explanation'>
```

In [38]:
```
person_index = 0  # Change only this: 0 = first customer in X_test, 1 = second,

shap.plots.waterfall(shap_values[person_index])
```



In [2]:
```
%pip install nbconvert pandas jinja2
```

```
Collecting nbconvert
  Downloading nbconvert-7.17.2-py3-none-any.whl.metadata (8.4 kB)
Requirement already satisfied: pandas in c:\RD\2026_2027\M3V2\.venv\Lib\site-pack
ages (3.0.6)
Collecting jinja2
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting beautifulsoup4 (from nbconvert)
  Downloading beautifulsoup4-4.15.0-py3-none-any.whl.metadata (3.8 kB)
Collecting bleach!=5.0.0 (from bleach[css]!=5.0.0->nbconvert)
  Downloading bleach-6.4.0-py3-none-any.whl.metadata (32 kB)
Collecting defusedxml (from nbconvert)
  Downloading defusedxml-0.7.1-py2.py3-none-any.whl.metadata (32 kB)
Requirement already satisfied: jupyter-core>=4.7 in c:\RD\2026_2027\M3V2\.venv\Li
b\site-packages (from nbconvert) (5.9.1)
Collecting jupyterlab-pygments (from nbconvert)
  Downloading jupyterlab_pygments-0.3.0-py3-none-any.whl.metadata (4.4 kB)
Collecting markupsafe>=2.0 (from nbconvert)
  Downloading markupsafe-3.0.4-cp312-cp312-win_amd64.whl.metadata (2.8 kB)
Collecting mistune<4,>=2.0.3 (from nbconvert)
  Downloading mistune-3.4.0-py3-none-any.whl.metadata (2.1 kB)
Collecting nbclient>=0.5.0 (from nbconvert)
  Downloading nbclient-0.11.0-py3-none-any.whl.metadata (7.3 kB)
Collecting nbformat>=5.7 (from nbconvert)
  Downloading nbformat-5.11.1-py3-none-any.whl.metadata (3.7 kB)
Requirement already satisfied: packaging in c:\RD\2026_2027\M3V2\.venv\Lib\site-p
ackages (from nbconvert) (26.3)
Collecting pandocfilters>=1.4.1 (from nbconvert)
  Downloading pandocfilters-1.5.1-py2.py3-none-any.whl.metadata (9.0 kB)
Requirement already satisfied: pygments>=2.4.1 in c:\RD\2026_2027\M3V2\.venv\Lib
\site-packages (from nbconvert) (2.21.0)
Requirement already satisfied: traitlets>=5.1 in c:\RD\2026_2027\M3V2\.venv\Lib\s
ite-packages (from nbconvert) (5.16.1)
Requirement already satisfied: numpy>=1.26.0 in c:\RD\2026_2027\M3V2\.venv\Lib\si
te-packages (from pandas) (2.5.4)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\RD\2026_2027\M3V2\.ve
nv\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\RD\2026_2027\M3V2\.venv\Lib\site-pack
ages (from pandas) (2026.5)
Collecting webencodings (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert)
  Downloading webencodings-0.6.1-py3-none-any.whl.metadata (3.4 kB)
Collecting tinycss2>=1.1.0 (from bleach[css]!=5.0.0->nbconvert)
  Downloading tinycss2-1.5.1-py3-none-any.whl.metadata (3.0 kB)
Requirement already satisfied: platformdirs>=2.5 in c:\RD\2026_2027\M3V2\.venv\Li
b\site-packages (from jupyter-core>=4.7->nbconvert) (4.13.5)
Requirement already satisfied: jupyter-client>=7.0.0 in c:\RD\2026_2027\M3V2\.ven
v\Lib\site-packages (from nbclient>=0.5.0->nbconvert) (8.10.0)
Requirement already satisfied: pyzmq>=25.0 in c:\RD\2026_2027\M3V2\.venv\Lib\site
-packages (from jupyter-client>=7.0.0->nbclient>=0.5.0->nbconvert) (27.2.0)
Requirement already satisfied: tornado>=6.4.1 in c:\RD\2026_2027\M3V2\.venv\Lib\s
ite-packages (from jupyter-client>=7.0.0->nbclient>=0.5.0->nbconvert) (6.5.10)
Requirement already satisfied: typing-extensions>=4.13.0 in c:\RD\2026_2027\M3V2
\.venv\Lib\site-packages (from jupyter-client>=7.0.0->nbclient>=0.5.0->nbconvert)
(4.16.0)
Collecting fastjsonschema>=2.15 (from nbformat>=5.7->nbconvert)
  Downloading fastjsonschema-2.22.2-py3-none-any.whl.metadata (2.1 kB)
Collecting jsonschema>=2.6 (from nbformat>=5.7->nbconvert)
  Downloading jsonschema-4.26.0-py3-none-any.whl.metadata (7.6 kB)
Collecting attrs>=22.2.0 (from jsonschema>=2.6->nbformat>=5.7->nbconvert)
  Downloading attrs-26.1.0-py3-none-any.whl.metadata (8.8 kB)
Collecting jsonschema-specifications>=2023.03.6 (from jsonschema>=2.6->nbformat>=
```

```
5.7->nbconvert)
  Downloading jsonschema_specifications-2025.9.1-py3-none-any.whl.metadata (2.9 k
B)
Collecting referencing>=0.28.4 (from jsonschema>=2.6->nbformat>=5.7->nbconvert)
  Downloading referencing-0.37.0-py3-none-any.whl.metadata (2.8 kB)
Collecting rpds-py>=0.25.0 (from jsonschema>=2.6->nbformat>=5.7->nbconvert)
  Downloading rpds_py-2026.9.1-cp312-cp312-win_amd64.whl.metadata (4.2 kB)
Requirement already satisfied: six>=1.5 in c:\RD\2026_2027\M3V2\.venv\Lib\site-pa
ckages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Collecting soupsieve>=1.6.1 (from beautifulsoup4->nbconvert)
  Downloading soupsieve-3.0.4-py3-none-any.whl.metadata (4.4 kB)
Downloading nbconvert-7.17.2-py3-none-any.whl (262 kB)
Downloading mistune-3.4.0-py3-none-any.whl (70 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading bleach-6.4.0-py3-none-any.whl (165 kB)
Downloading markupsafe-3.0.4-cp312-cp312-win_amd64.whl (14 kB)
Downloading nbclient-0.11.0-py3-none-any.whl (25 kB)
Downloading nbformat-5.11.1-py3-none-any.whl (79 kB)
Downloading fastjsonschema-2.22.2-py3-none-any.whl (27 kB)
Downloading jsonschema-4.26.0-py3-none-any.whl (90 kB)
Downloading attrs-26.1.0-py3-none-any.whl (67 kB)
Downloading jsonschema_specifications-2025.9.1-py3-none-any.whl (18 kB)
Downloading pandocfilters-1.5.1-py2.py3-none-any.whl (8.7 kB)
Downloading referencing-0.37.0-py3-none-any.whl (26 kB)
Downloading rpds_py-2026.9.1-cp312-cp312-win_amd64.whl (222 kB)
Downloading tinycss2-1.5.1-py3-none-any.whl (28 kB)
Downloading webencodings-0.6.1-py3-none-any.whl (8.7 kB)
Downloading beautifulsoup4-4.15.0-py3-none-any.whl (109 kB)
Downloading soupsieve-3.0.4-py3-none-any.whl (42 kB)
Downloading defusedxml-0.7.1-py2.py3-none-any.whl (25 kB)
Downloading jupyterlab_pygments-0.3.0-py3-none-any.whl (15 kB)
Installing collected packages: webencodings, soupsieve, rpds-py, pandocfilters, m
istune, markupsafe, jupyterlab-pygments, fastjsonschema, defusedxml, attrs, tinyc
ss2, referencing, jinja2, bleach, beautifulsoup4, jsonschema-specifications, json
schema, nbformat, nbclient, nbconvert

   -- ---------------------------------------  1/20 [soupsieve]
   ------ ---------------------------------  3/20 [pandocfilters]
   -------- -------------------------------  4/20 [mistune]
   -------- -------------------------------  4/20 [mistune]
   -------- -------------------------------  4/20 [mistune]
   -------- -------------------------------  4/20 [mistune]
   -------- -------------------------------  4/20 [mistune]
   -------- -------------------------------  4/20 [mistune]
   ------------ ---------------------------  6/20 [jupyterlab-pygments]
   -------------- -------------------------  7/20 [fastjsonschema]
   ---------------- -----------------------  8/20 [defusedxml]
   ------------------ ---------------------  9/20 [attrs]
   ------------------ ---------------------  9/20 [attrs]
   ------------------ ---------------------  9/20 [attrs]
   -------------------- ------------------- 10/20 [tinycss2]
   ---------------------- ----------------- 11/20 [referencing]
   ---------------------- ----------------- 11/20 [referencing]
   ----------------------- ---------------- 12/20 [jinja2]
   ----------------------- ---------------- 12/20 [jinja2]
   ----------------------- ---------------- 12/20 [jinja2]
   ----------------------- ---------------- 12/20 [jinja2]
   ------------------------- -------------- 13/20 [bleach]
   ------------------------- -------------- 13/20 [bleach]
   ------------------------- -------------- 13/20 [bleach]
```

```
   ------------------------- ------------- 13/20 [bleach]
   ------------------------- ------------- 13/20 [bleach]
   ------------------------- ------------- 13/20 [bleach]
   --------------------------- ------------ 14/20 [beautifulsoup4]
   --------------------------- ------------ 14/20 [beautifulsoup4]
   --------------------------- ------------ 14/20 [beautifulsoup4]
   ----------------------------- ---------- 15/20 [jsonschema-specifications]
   ------------------------------- -------- 16/20 [jsonschema]
   ------------------------------- -------- 16/20 [jsonschema]
   ------------------------------- -------- 16/20 [jsonschema]
   ------------------------------- -------- 16/20 [jsonschema]
   ------------------------------- -------- 16/20 [jsonschema]
   --------------------------------- ------ 17/20 [nbformat]
   --------------------------------- ------ 17/20 [nbformat]
   --------------------------------- ------ 17/20 [nbformat]
   --------------------------------- ------ 17/20 [nbformat]
   --------------------------------- ------ 17/20 [nbformat]
   ----------------------------------- ---- 18/20 [nbclient]
   ----------------------------------- ---- 18/20 [nbclient]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 19/20 [nbconvert]
   ------------------------------------ --- 20/20 [nbconvert]

Successfully installed attrs-26.1.0 beautifulsoup4-4.15.0 bleach-6.4.0 defusedxml
-0.7.1 fastjsonschema-2.22.2 jinja2-3.1.6 jsonschema-4.26.0 jsonschema-specificat
ions-2025.9.1 jupyterlab-pygments-0.3.0 markupsafe-3.0.4 mistune-3.4.0 nbclient-
0.11.0 nbconvert-7.17.2 nbformat-5.11.1 pandocfilters-1.5.1 referencing-0.37.0 rp
ds-py-2026.9.1 soupsieve-3.0.4 tinycss2-1.5.1 webencodings-0.6.1
Note: you may need to restart the kernel to use updated packages.
```